RAG на практике и введение в агентов

План

- RAG с нуля: архитектура и первый рабочий пайплайн
- RAG в продакшене: переключение на Qdrant и асинхронность
- Оценка качества RAG
- Защита от галлюцинаций и фильтрация ответов
- Введение в агентов: LLM + инструменты (Tools)
- Stateful-агенты с LangGraph
- Подготовка к продакшену: логирование, мониторинг, безопасность

❑RAG с нуля: архитектура и первый рабочий пайплайн

RAG (Retrieval-Augmented Generation) решает проблему ограниченного контекста и устаревших знаний LLM, подставляя в промпт релевантные фрагменты из внешней базы знаний.

Архитектура пайплайна:

- **Indexing** (Индексация): Документ → Чанки → Эмбеддинги → Векторная БД.
- **Retrieval** (Поиск): Запрос пользователя → Эмбеддинг запроса → Поиск топ-K похожих чанков.
- **Generation** (Генерация): Системный промпт + Найденные чанки + Запрос пользователя → LLM → Ответ.

RAG в продакшене: переключение на Qdrant и асинхронность

В продакшене система должна обрабатывать сотни запросов в секунду. Синхронные вызовы блокируют поток, а in-memory базы не масштабируются.

Ключевые изменения:

- Использование Qdrant (или Milvus/Pinecone) как выделенного сервиса.
- Использование asyncio и асинхронных клиентов для неблокирующего ввода-вывода (I/O).

Оценка качества RAG

Как понять, что RAG работает хорошо? Субъективной оценки недостаточно. Используются фреймворки типа RAGAS или TruLens, которые применяют подход LLM-as-a-Judge (LLM как судья).

Три ключевые метрики:

- **Faithfulness** (Верность): Насколько ответ LLM опирается только на извлеченный контекст (защита от галлюцинаций).
- **Answer Relevance** (Релевантность ответа): Насколько ответ напрямую отвечает на исходный вопрос пользователя.
- **Context Precision** (Точность контекста): Насколько высоко в выдаче векторной БД оказались релевантные чанки (оценка качества поиска)

Оценка качества RAG

Пример промпта для оценки Faithfulness (LLM-as-a-Judge):

judge_prompt = f"""

Оцени, основан ли ответ ИСКЛЮЧИТЕЛЬНО на предоставленном контексте.

Контекст: {context}

Ответ: {generated_answer}

Оцени по шкале от 0 до 1:

1.0: Ответ полностью следует контексту, нет выдуманных фактов.

0.0: Ответ содержит галлюцинации или информацию, которой нет в контексте.

Твой ответ (только число):

"""

Запускаем этот промпт через сильную модель (например, GPT-4o или локальную GPT-4-уровня)

Защита от галлюцинаций и фильтрация ответов

Даже с RAG модели склонны "додумывать". Стратегии защиты:

- Строгий System Prompt: (как в примере выше) + требование указывать источники.

Пример: "В конце каждого предложения, основанного на контексте, ставь ссылку на источник, например [doc_1]."

- Citation Checking (Проверка цитирования): Отдельный легкий шаг после генерации, где другая (или та же) модель проверяет, действительно ли в указанном чанке есть эта информация.
- Thresholding (Пороговое значение): Если максимальная оценка схожести (score) от векторной БД или реранкера ниже порога (например, Cosine Similarity < 0.65), мы вообще не отправляем запрос в LLM, а сразу возвращаем: "Информация не найдена".

Введение в агентов: LLM + инструменты (Tools)

RAG пассивен: он только ищет и отвечает. Агент активен: он рассуждает, решает, какой инструмент использовать, вызывает его, анализирует результат и повторяет цикл (паттерн ReAct: Reason + Act).

Инструмент (Tool) — это любая Python-функция, описание которой (name, description, args) передается в LLM.

Введение в агентов: LLM + инструменты (Tools)

```
# Пример определения инструмента (концептуально, синтаксис LangChain)

from langchain_core.tools import tool

@tool

def get_current_weather(location: str, unit: str = "celsius") -> str:

    """Получает текущую погоду в указанном городе. Используй этот инструмент, когда пользователь спрашивает о погоде."""

    # В реальности здесь был бы запрос к API погоды

    return f"В городе {location} сейчас 22 градуса {unit}."


# LLM получает список доступных инструментов.

# Если пользователь спрашивает "Какая погода в Москве?", LLM не отвечает сразу,

# а генерирует вызов: call: get_current_weather(location="Москва")
```

Stateful-агенты с LangGraph

Стандартные агенты (как например в LangChain) — это "черный ящик" с циклом while. LangGraph (от создателей LangChain) позволяет строить агентов как ориентированный граф, где есть явное управление состоянием (State) и циклами. Это необходимо для сложных сценариев с памятью и человеческим вмешательством (Human-in-the-loop).

Почему LangGraph? Проблема «черного ящика»

В классических фреймворках (например, AgentExecutor в старом LangChain) агент работал по простому циклу:

```
while True:
    action = llm.decide_next_action(history)
    if action == "finish": break
    observation = execute_tool(action)
    history.append(observation)
```

Почему LangGraph? Проблема «черного ящика»

Проблемы этого подхода:

Отсутствие явного состояния (State): История сообщений просто лежит в списке. Если вы хотите добавить в контекст что-то еще (например, промежуточные вычисления, метаданные пользователя, флаги), вам приходится «хаковать» промпт.

Нет циклов и ветвлений: Сложная бизнес-логика (например, «если клиент VIP, идем в ветку А, если нет — в ветку Б, а если возникла ошибка, возвращаемся на шаг 1») описывается костылями внутри одного узла.

Невозможность паузы (Human-in-the-loop): Вы не можете остановить агента перед выполнением опасного инструмента (например, delete_database), показать результат человеку, дождаться правки и продолжить с того же места.

LangGraph решает это, представляя агента не как цикл, а как ориентированный граф (часто с циклами), где явно определены Состояние (State), Узлы (Nodes) и Ребра (Edges).

Три кита LangGraph: State, Nodes, Edges

1. **State (Состояние) — «Общая доска»**

State — это объект (обычно TypedDict или Pydantic-модель), который передается от узла к узлу. Это единственный источник истины в графе.

Узлы не общаются друг с другом напрямую. Они читают из State и возвращают обновления (updates) для State.

Reducers (Редуцеры): Критически важная концепция. Если в State есть список messages, как новый узел должен его обновить? Заменить целиком? Или добавить свои сообщения в конец? За это отвечают аннотации (например, Annotated[list, add_messages]).

Три кита LangGraph: State, Nodes, Edges

2. Nodes (Узлы) — «Рабочие»

Узел — это обычная Python-функция (или LangChain Runnable), которая принимает текущий State и возвращает словарь с новыми значениями.

Примеры узлов: call_llm, execute_tool, format_output, check_permissions.

Три кита LangGraph: State, Nodes, Edges

C. Edges (Ребра) — «Маршрутизация»

Ребра определяют, куда граф пойдет после завершения узла.

Normal Edge: Жесткий переход (А → Б).

Conditional Edge: Переход на основе текущего State. Функция-роутер смотрит в State и возвращает имя следующего узла. (Например: «Если в сообщении есть tool_calls, идем в tool_node, иначе в END»).

Три кита LangGraph: State, Nodes, Edges



Подготовка к продакшену: логирование, мониторинг, безопасность

Система не готова к продакшену, если мы не знаем, что в ней происходит.

- Логирование и трассировка (Tracing): Использование LangSmith, Arize Phoenix или OpenTelemetry. Они записывают каждый шаг: сырой промпт, извлеченный контекст, время выполнения (latency), потребленные токены и финальный ответ.

- Мониторинг: Дашборды с метриками:

- P95 Latency (время ответа для 95% пользователей).

- Rate of "I don't know" responses (если слишком много, значит, база знаний плохая или чанкинг неверный).

Подготовка к продакшену: логирование, мониторинг, безопасность

Безопасность (LLM Security):

- PII Redaction: Использование библиотек типа Microsoft Presidio для автоматического удаления имен, телефонов и паспортов из запроса пользователя перед отправкой в LLM.
- Guardrails: Фреймворки вроде NeMo Guardrails для жесткого ограничения тем, которые может обсуждать бот (например, запрет на обсуждение политики или конкурентов).

RAG Architecture

